

Senior Product Engineer

Round 3: Take-Home Exercise & Roundtable Interview

Overview

This final round consists of two parts: a take-home exercise completed on your own time, followed by a roundtable interview with our engineering team.

Component	Duration	Format
Take-Home Exercise	2-3 hours	Independent work, any resources allowed
Roundtable Interview	60-90 minutes	Presentation, code review, discussion

Part 1: Take-Home Exercise

The Prompt

Context

Parking enforcement officers spend their shifts walking or driving between zones, checking for violations. Currently they use paper lists and radio dispatch. You're prototyping a mobile-first companion app to modernize their workflow.

Your Task

Build a prototype that helps officers:

- Know where to go (which zones need attention)
- Understand what's happening at a zone (current vehicles, violation status)
- Log their activity (visited zone, issued warning, etc.)

You may not have a fully polished product to present—that's intentional. Your job is to:

1. Decide which features are most important for an MVP
2. Build those features well enough to demonstrate the concept
3. Design a mock API that your frontend consumes
4. Handle the real-world messiness: loading states, error states, and edge cases

Technical Requirements

- **Mobile-first design:** 375px minimum viewport width

- **Mock API layer:** Design and implement mock API endpoints that your frontend calls. Document the API contract (endpoints, methods, request/response shapes). Use any mocking approach you prefer (MSW, json-server, Next.js API routes, hard-coded fetch wrappers with simulated delay, etc.)—but the frontend should call functions or endpoints that resemble real API calls, not import JSON directly.
- **State handling:** Your UI must gracefully handle loading states (data in flight), error states (API failures, network issues), and empty states (no data to display).
- **Framework:** Any frontend framework you're comfortable with
- **Design:** You own all visual/UX design—no mockups will be provided

Available Data Concepts

You may assume backend systems could provide data about:

- Zones (id, name, location, current occupancy, max capacity, violation count)
- Vehicles (id, zone, type, arrival time, time limit, overstay status)
- Alerts/violations (id, zone, vehicle, severity, timestamp)
- Officer activity log (actions taken, timestamps)

How you structure this into API endpoints is part of the exercise.

Deliverables

1. **Git repository** (GitHub, GitLab, or similar) with a clear README containing:

- Setup and run instructions
- Tech stack choices and rationale
- API documentation (endpoints, request/response examples)
- How to trigger error states for demo purposes (e.g., query param, toggle, or documented instructions)

2. **Runnable locally**

3. **Written summary** (500-750 words) covering:

- What did you prioritize and why?
- What did you cut and why?
- What assumptions did you make about user needs?
- How did you approach the API design? What tradeoffs did you make?
- What would you build next with another 2-3 hours?

Time Expectation

2-3 hours. We value your time—please don't feel obligated to exceed this significantly. We're evaluating your decision-making and prioritization as much as your output.

You may use any resources at your disposal, including AI tools, documentation, and libraries.

Submission

Reply to the invitation email with a link to your repository **at least 24 hours before** your scheduled interview time.

Part 2: Roundtable Interview

Once we receive your submission, you'll join a 60-90 minute roundtable interview with several members of our engineering team.

Interview Format

Segment	Duration	Description
Candidate Presentation	15-20 min	You'll walk us through what you built, demo the app, and explain your decisions
Code Review	25-30 min	We'll look at specific parts of your code together and discuss your approach
Design & Product Discussion	15-20 min	We'll dig deeper into your UX decisions and tradeoffs
Your Questions	5-10 min	Time for you to ask us anything about the team or role

How to Prepare

- Be ready to **share your screen** and demo your running application
- Be prepared to **navigate your codebase** and discuss specific files or functions
- Think through the **tradeoffs** you made—we'll ask about alternatives you considered
- Review your **written summary**—it will guide part of our conversation

We're not looking for perfection. We're looking for thoughtful decisions, clear communication, and the ability to discuss your work openly—including its limitations.

Evaluation Criteria

We will assess your work across the following dimensions:

Dimension	What We Look For
Product Thinking	Did you prioritize features that matter for a field officer? Do you understand the user?
UI/UX Design	Is it usable on mobile? Does it feel intentional? Is the visual hierarchy clear?
Code Quality	Is it readable, organized, and maintainable? Appropriate separation of concerns?
API Design	Are the endpoints sensible and well-documented? Would this scale to a real backend?
Resilience	Do loading, error, and empty states feel considered—not bolted on?
Communication	Can you articulate your reasoning clearly, both in writing and verbally?

Detailed Evaluation Rubric

Dimension	Strong Signal	Weak Signal
Prioritization	Builds 2-3 coherent features that clearly serve field officers and state troopers. No clear endgame. Cuts feel accidental.	Builds 2-3 unrelated features. No clear endgame. Cuts feel accidental.
API Design	Endpoints map to user tasks. Payloads are minimal and single purposeful endpoints. No documentation. Overly complex.	Endpoints map to user tasks. Payloads are minimal and single purposeful endpoints. No documentation. Overly complex.
Loading States	Contextual loading indicators. Skeleton screens or spinners. Page doesn't look like anything. Overly complex.	Contextual loading indicators. Skeleton screens or spinners. Page doesn't look like anything. Overly complex.
Error States	User-friendly messages. Retry options where sensible. Console errors. Easy to demo.	User-friendly messages. Retry options where sensible. Console errors. Easy to demo.
Empty States	Helpful empty states ("No violations—you're all caught up").	Helpful empty states ("No violations—you're all caught up").
Mobile UX	Touch-friendly targets. Thumb-zone awareness. Scannable. A glance needed. Works on mobile.	Touch-friendly targets. Thumb-zone awareness. Scannable. A glance needed. Works on mobile.
Visual Design	Consistent spacing, typography, color. Feels intentional. Consistent styling. High visual hierarchy. Looks like unstyled HTML or generic.	Consistent spacing, typography, color. Feels intentional. Consistent styling. High visual hierarchy. Looks like unstyled HTML or generic.
Code Organization	Clear separation of concerns. API layer abstracted. Components are reused through components. Giant files. No discernible structure.	Clear separation of concerns. API layer abstracted. Components are reused through components. Giant files. No discernible structure.
Written Summary	Specific reasoning. Acknowledges tradeoffs. Shows awareness of what's possible. Justification. Reads like AI-generated filler.	Specific reasoning. Acknowledges tradeoffs. Shows awareness of what's possible. Justification. Reads like AI-generated filler.

Questions?

If anything about the exercise is unclear, please don't hesitate to reach out. Clarifying questions are welcome and will not affect your evaluation.

We're excited to see what you build!